

Database Management System

LAB Report

Students Name:

- **Arghajit Das (11200123007)**
- **Bhaskar Roy (11200123009)**
- **Budhaditya Sanyal (11200123011)**
- **Debarnab Das (11200123012)**
- **Dibyanand Mishra (11200123013)**

Semester : Sixth

Stream : Computer Science and Engineering

Batch : 2023 - 2027

Government College of Engineering and Leather Technology

Acknowledgement

We would like to express our profound gratitude to our esteemed professor, **Prof. Srirupa Dasgupta**, for her invaluable guidance, continuous encouragement, and constructive feedback throughout the course of this Database Management Systems laboratory project. Her expertise and clear academic direction provided the essential theoretical foundation and practical insights required for designing and implementing this complex Three-Tier Hotel Booking architecture.

We also extend our sincere thanks to the Department of Computer Science and Engineering at the **Government College of Engineering and Leather Technology (GCELT)**, affiliated with Maulana Abul Kalam Azad University of Technology (MAKAUT). The department provided us with the necessary laboratory resources, technical infrastructure, and a highly conducive academic environment that made the successful completion of this practical assignment possible.

Furthermore, we wish to acknowledge the collaborative spirit and dedicated effort of every group member. Building a decoupled web application required seamless coordination between designing the relational schema, developing the FastAPI backend, and constructing the React.js frontend. The successful integration of these systems stands as a testament to our collective teamwork and shared dedication.

Finally, we are grateful to our peers, lab assistants, and everyone who directly or indirectly extended their support and shared their knowledge during the development and testing phases of this project.

INDEX

Section	Topic / Title	Page No.
1.	Aim & Objective	1
2.	Problem Statement	1
3.	System Architecture	1
4.	Database Design	2
4.1	Entity Relationships	2
4.2	Entity-Relationship (ER) Diagram	2
4.3	Logical Schema Diagram	2
5.	Database Implementation (DDL & DML)	3
6.	Application Logic & Query Implementation	4
7.	Project Evaluation	5
8.	Conclusion	5

EXPERIMENT NO.: 16

1. Aim & Objective

To design, implement, and integrate a Database Management System (DBMS) for a Hotel Booking application using SQL and a Three-Tier (T3) web architecture.

2. Problem Statement

Develop a DBMS for Hotel Booking using SQL queries on a chosen DBMS platform with a frontend interface.

Tables & Attributes:

- Hotels: Hotel_ID (PK), Hotel_Name, City, Star_Rating
- Rooms: Room_ID (PK), Hotel_ID (FK), Room_No, Room_Type, Price_Per_Night, Availability
- Reservations: Reservation_ID (PK), Room_ID (FK), Guest_Name, Check_In_Date, Check_Out_Date

The System must fulfill the following functional requirements:

- Store records for Hotels, Rooms, and Reservations.
- Fetch hotel names located within a specific city.
- List rooms that are currently marked as available.
- Check active reservation windows using date criteria.
- Process new room bookings safely using transactions.

3. System Architecture

The project is built using a **Three-Tier (T3) Architecture**:

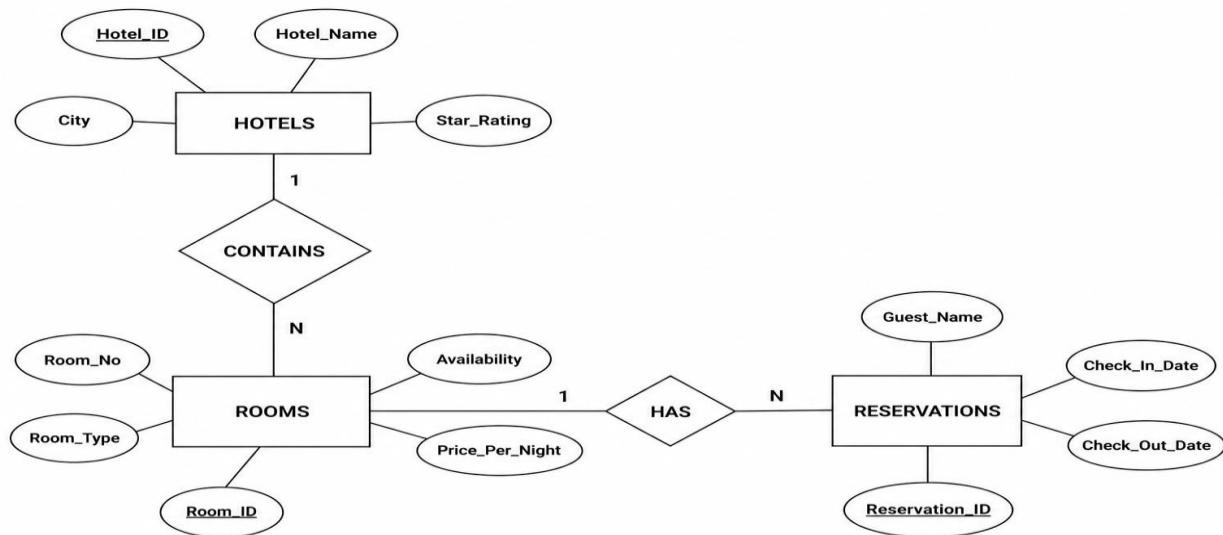
1. **Presentation Tier (Frontend):** A decoupled React.js user interface.
2. **Application Tier (Backend):** A Python FastAPI server that acts as the middle layer, executing business logic and preventing direct client access to the database.
3. **Data Tier (Database):** A MySQL RDBMS storing the data and enforcing integrity constraints

4. Database Design

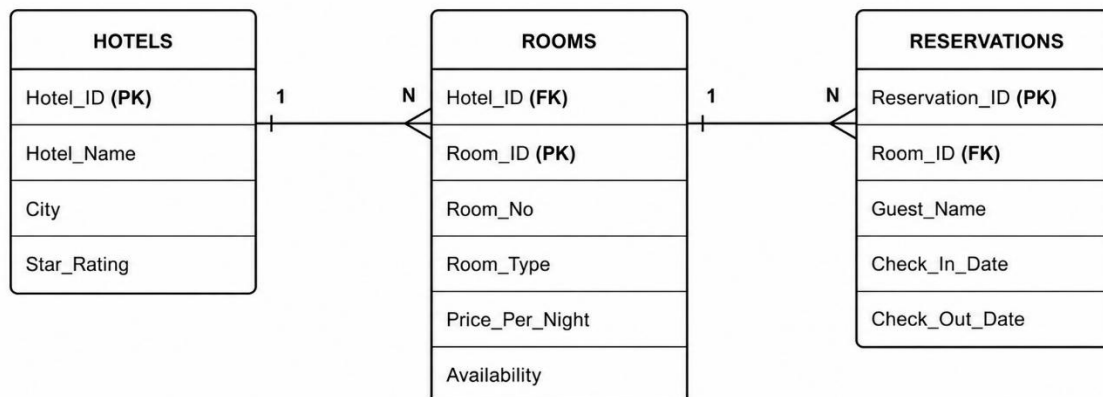
4.1 The database consists of three primary entities with the following relationships:

- A One-to-Many relationship between Hotels and Rooms (A hotel can have multiple rooms).
- A One-to-Many relationship between Rooms and Reservations (A room can have multiple reservations over time).

4.2 Entity-Relationship (ER) Diagram The conceptual model consists of three main entities with One-to-Many relationships.



4.3 Logical Schema Diagram The relational model maps the entities into tables, highlighting Primary Keys (PK) and Foreign Keys (FK) used to enforce referential integrity.



5. Database Implementation (DDL & DML)

Data Definition Language (DDL) was used to define the relation schemas, and Data Manipulation Language (DML) was used to populate the initial state.

The database structure and initial mock data were established using the following SQL script:

SQL

```
DROP DATABASE IF EXISTS HotelBookingDB;
CREATE DATABASE IF NOT EXISTS HotelBookingDB;
USE HotelBookingDB;

CREATE TABLE Hotels (
    Hotel_ID INT PRIMARY KEY,
    Hotel_Name VARCHAR(255) NOT NULL,
    City VARCHAR(100) NOT NULL,
    Star_Rating INT CHECK (Star_Rating BETWEEN 1 AND 5)
);

CREATE TABLE Rooms (
    Room_ID INT PRIMARY KEY,
    Hotel_ID INT,
    Room_No VARCHAR(10) NOT NULL,
    Room_Type VARCHAR(50),
    Price_Per_Night DECIMAL(10, 2),
    Availability BOOLEAN DEFAULT 1,
    FOREIGN KEY (Hotel_ID) REFERENCES Hotels(Hotel_ID) ON DELETE CASCADE
);

CREATE TABLE Reservations (
    Reservation_ID INT PRIMARY KEY,
    Room_ID INT,
    Guest_Name VARCHAR(255) NOT NULL,
    Check_In_Date DATE,
    Check_Out_Date DATE,
    FOREIGN KEY (Room_ID) REFERENCES Rooms(Room_ID) ON DELETE CASCADE
);

-- Mock Data Insertion
INSERT INTO Hotels VALUES (1, 'Grand Palace', 'Kolkata', 5), (2, 'Sea View Inn', 'Mumbai', 3);
INSERT INTO Rooms VALUES (101, 1, '10A', 'Deluxe', 5000.00, 1), (102, 1, '10B', 'Suite', 8000.00, 0);
INSERT INTO Reservations VALUES (1001, 102, 'Rahul Sharma', '2026-07-10', '2026-07-15');
```

6. Application Logic & Query Implementation

The application interacts with the database via an API, utilizing Data Retrieval Language (DRL) and DML for core operations.

Query 1: Fetch Hotels by City

- **Operation:** Filters the Hotels table using the WHERE clause based on the City attribute.

```
SQL
SELECT Hotel_Name
FROM Hotels
WHERE City = 'Kolkata';
```

Query 2: List Available Rooms

- **Operation:** Retrieves rooms where the Availability boolean is set to 1 (True).

```
SQL
SELECT Room_ID, Room_No, Room_Type, Price_Per_Night
FROM Rooms
WHERE Availability = 1;
```

Query 3: Check Active Reservations (Using JOINS)

- **Operation:** Utilizes INNER JOIN operations to cross-reference data between Reservations, Rooms, and Hotels, returning a combined dataset filtered by date ranges.

```
SQL
SELECT r.Reservation_ID, r.Guest_Name, r.Check_In_Date,
r.Check_Out_Date, h.Hotel_Name, h.City
FROM Reservations r
JOIN Rooms rm ON r.Room_ID = rm.Room_ID
JOIN Hotels h ON rm.Hotel_ID = h.Hotel_ID
WHERE r.Check_In_Date <= '2026-07-15' AND r.Check_Out_Date >=
'2026-07-10';
```

Query 4: Room Booking Transaction (Advanced Implementation)

- **Operation:** A compound operation requiring ACID properties. It checks availability, performs an INSERT into Reservations, and executes an UPDATE on Rooms. The transaction is finalized using COMMIT, or safely reversed using ROLLBACK if an error occurs.

```
SQL
START TRANSACTION;
INSERT INTO Reservations (Reservation_ID, Room_ID, Guest_Name,
Check_In_Date, Check_Out_Date)
VALUES (1009, 101, 'Debarnab Das', '2026-08-01', '2026-08-05');
UPDATE Rooms
SET Availability = 0
WHERE Room_ID = 101;
COMMIT;
```

7. Project Evaluation

Advantages:

- **Security:** Utilizing a T3 architecture prevents SQL injection and hides database credentials from the client side.
- **Data Integrity:** The use of `COMMIT` and `ROLLBACK` guarantees ACID properties during the booking transaction.
- **Efficiency:** The implementation of SQL `JOIN` clauses reduces the number of API calls required by the frontend.

Limitations:

- **Availability Logic:** The current schema utilizes a static 1 or 0 for room availability. It does not account for dynamic date-range availability (e.g., booking a room for next month makes it unavailable for today).

Future Scope:

- Restructure the database schema to calculate room availability dynamically by querying overlapping dates directly from the `Reservations` table rather than relying on a static boolean in the `Rooms` table.

8. Conclusion

The experiment successfully demonstrated the practical application of Relational Database concepts through the end-to-end development of a Hotel Booking system. A robust database schema was designed, implemented in MySQL, and securely interfaced with a modern Three-Tier (T3) web application stack. By successfully decoupling the React.js presentation layer from the MySQL data tier using a Python FastAPI middle layer, the project bridged the gap between theoretical database administration and real-world software engineering.

The system fulfilled all problem statement requirements, efficiently executing complex SQL queries, including multi-table `JOIN` operations, to retrieve active reservations, available rooms, and city-specific hotel data. Furthermore, the development process provided valuable hands-on insights into enforcing referential integrity via Primary and Foreign Key cascading constraints. Most importantly, implementing the room booking mechanism reinforced the critical necessity of ACID properties, utilizing `COMMIT` and `ROLLBACK` commands to ensure transaction safety and data consistency. Ultimately, this laboratory project served as a highly effective exercise in designing scalable, secure, and robust database architectures for modern web applications.